PENSE-BÊTE · TECHNIQUE ET COMMANDES

# Architecture et gestion des processus Windows Server

06-230 PSE 25-27 2-7 Architecture et gestion des processus dans Windows Server — source : fiche de révision du module (gemma4, 03/11/2026)

**À RETENIR**

1. Le processus assure une isolation stricte via un espace mémoire virtuel indépendant avec un adressage 64 bits.
2. Le thread constitue l'unité fondamentale d'exécution dont le temps d'exécution est défini par le Quantum.
3. L'architecture NUMA et le Soft NUMA sont critiques pour la performance des serveurs à haute densité.

## Les processus sous Windows Server

| Notion | À retenir |
|---|---|
| Définition et rôle | Assure une **isolation stricte** via un **espace mémoire virtuel indépendant** et des **handles** vers des ressources. Sécurité : segmentation **ring 3** (utilisateur) et **ring 0** (noyau). |
| Espace d'adressage et cycle de vie | Adressage **64 bits** obligatoire. Espace virtuel peut dépasser **128 To** selon les éditions. Supervision par le **Service Control Manager** pour redémarrage automatique. |
| Les Job Objects | Isolation type **cgroups Linux**. Permettent : **CPU rate control**, plafonds **mémoire**, limites **E/S par seconde** et **comptabilisation complète** des ressources du groupe. |

## Threads et exécution parallèle

| Notion | À retenir |
|---|---|
| Gestion des Threads | Unité fondamentale d'exécution (**centaines** possibles) ; optimisation sur architectures à **grand nombre de cœurs** et équilibrage **NUMA** ; gestion des threads inactifs pour réduire la **consommation énergétique**. |
| Le Pool de Thread moderne | Gère automatiquement le **nombre optimal** de threads ; intégration complète avec les ports **IOCP** ; mode **adaptatif** pour les charges de travail de type **cloud**. |
| User Mode Scheduling et Fibres | **User Mode Scheduling** : permet aux applications haute performance (ex: **bases de données**) de gérer leur propre ordonnancement ; **Fibres** : technologie **obsolète** et **déconseillée** (incompatibles avec la **sécurité moderne**). |

## Communication inter-processus

| Notion | À retenir |
|---|---|
| Mécanismes classiques | Utilisation de **Named Pipes** (SMB/IIS), **Sockets** (TCP Fast Open, IPv6, SMB sur QUIC), **fichiers mappés en mémoire** (NUMA) et **objets de synchronisation** (mutex, sémaphores). |
| Mécanismes modernes et obsolètes | **RPC** (sécurité renforcée), **gRPC** (.NET/conteneurs), **VMBus** (VM/hôte) ; **Mailslots** et **NetBIOS** sont obsolètes. |

## Architecture NUMA

| Notion | À retenir |
|---|---|
| Rôle et importance | Gère les différences de latence d'accès à la mémoire selon sa localisation physique pour les serveurs à plusieurs processeurs et nœuds mémoire. |
| Soft NUMA | Définit des nœuds logiques pour subdiviser les ressources matérielles ; améliore la scalabilité et la répartition des charges (**IIS**, **SQL Server**). |
| Gestion et APIs | Windows optimise l'affinité des threads et l'allocation de mémoire locale. APIs : `GetNumaNodeProcessorMaskEx`, `VirtualAlloc2` et `SetThreadGroupAffinity` (groupes de processeurs). |

## Multitâche et ordonnancement

| Notion | À retenir |
|---|---|
| Principes du multitâche | Système **préemptif** : le noyau peut interrompre un thread à tout moment. États : **Prêt**, **En cours d'exécution**, **En attente**, **Suspendu** et **Terminé**. Les **wake-ups** sont accélérés. |
| L'Ordonnanceur | Gère les groupes de processeurs pour **> 64** processeurs logiques. Prend en compte **Hyper-Threading**, placement **NUMA** et **consommation énergétique**. Files d'attente par processeur et groupe. |
| Le Quantum | Temps d'exécution attribué à un thread, défini **dynamiquement**. S'adapte aux **charges du serveur** et diffère selon les **classes de services**. |

## Priorités et ajustements

| Notion | À retenir |
|---|---|
| Modèle de priorités | Échelle de **0 à 31** : niveaux **0 à 15** pour threads **normaux**, niveaux **16 à 31** pour threads **temps réel**. |
| Ajustements dynamiques | Augmentation temporaire des threads **interactifs** ; mécanisme contre la **famine** ; **Priority Boost** augmente la priorité des processus en **attente prolongée**. |

## Influence de la virtualisation

| Notion | À retenir |
|---|---|
| Interaction Hyper-V et Processus | Les processus partagent les **cœurs physiques** avec d'autres **machines virtuelles**. L'hyperviseur peut **suspendre des threads** pour préserver l'**isolation** entre les environnements. |
| Enlightened Scheduling et VMBus | Collaboration pour optimiser le **placement des VM** et l'accès aux ressources. Le **temps CPU réel** est pris en compte. Le **VMBus** facilite la communication (hôte/VM et VM/VM). |

## Mitigations et Contrôles

- **Control Flow Guard (CFG)** : protège le flux de contrôle du programme.
- **Data Execution Prevention (DEP)** : empêche l'exécution de code dans certaines zones mémoire.
- **ASLR** : randomise l'agencement de l'espace d'adressage pour compliquer les attaques.
- **SEHOP** : renforcé pour protéger contre l'écrasement du gestionnaire d'exceptions.
- **Contrôle des privilèges et isolation par tokens basse** : renforcés.

## Pièges et confusions

⚠ Confondre les fibres avec un mécanisme de gestion moderne alors qu'elles sont obsolètes et dangereuses pour la sécurité.
⚠ Utiliser NetBIOS au lieu de mécanismes modernes comme RPC ou gRPC.
⚠ Oublier que les Job Objects offrent une isolation des ressources comparable aux cgroups sous Linux.
⚠ Ignorer que l'utilisation de Mailslots est désormais obsolète.

## Sigles et définitions

**Processus** Instance en cours d'exécution d'un programme représentant une entité d'isolation avec un espace mémoire virtuel indépendant.
**Thread** Unité fondamentale d'exécution contenue dans un processus.
**Job Object** Objet noyau permettant de regrouper des processus pour leur appliquer des contraintes de ressources et gérer leur cycle de vie.
**NUMA** Architecture système où le temps d'accès à la mémoire dépend de la localisation physique de la mémoire par rapport au processeur.
**Quantum** Temps d'exécution alloué à un thread par l'ordonnanceur avant qu'il ne puisse être préempté.
**IOCP** Modèle d'entrée et sortie asynchrone performant intégré au pool de threads moderne.
**VMBus** Canal de communication haute performance entre l'hôte et les machines virtuelles ou entre les machines virtuelles elles-mêmes.

## Auto-test

1. Quelle est l'unité fondamentale d'exécution contenue dans un processus ?
2. Quel mécanisme permet de regrouper des processus pour appliquer des contraintes de ressources ?
3. Comment appelle-t-on le temps d'exécution alloué à un thread par l'ordonnanceur ?
4. Quelle technologie de communication est utilisée entre l'hôte et les machines virtuelles ?
5. Quelle est l'échelle des niveaux de priorité pour les threads ?

Réponses : **1.** Le thread · **2.** Le Job Object · **3.** Le Quantum · **4.** Le VMBus · **5.** De 0 à 31

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.